

Udhëzues për Dokumentim dhe Prezantim - 40 Pikë

Punuar nga: Dion Salihu

Projekti: Sistem mikrosërbbimesh me autentifikim JWT dhe modul AI

1. Përmbledhja e Projektit

Ky dokument shpjegon implementimin e projektit në versionin e tij 40-pikësh. Ky projekt përfshin:

- Auth Service me bcrypt dhe JWT
- 2 Resource Services që validon JWT
- AI Module që trajnon Isolation Forest dhe ruan modelin
- Dokumentim të përgjigjeve të kërkuara

Kërkesat e avancuara si TLS/Wireshark, bllokim real-time, simulator brute-force, impossible travel, dashboard me SignalR dhe false-positive tuning nuk janë pjesë e këtij versioni 40-pikësh.

2. Përditësimet e nevojshme për mjedisin tuaj

Sistemi funksionon në mjedisin tuaj lokal me:

- dotnet 10.0.107
- docker compose up -d postgres-db
- ./scripts/run-all-services.sh

Për shkak të një PostgreSQL lokal të përdorur në portin 5432 në makinën tuaj, projekti tani përdor portin **5433** për Docker PostgreSQL. Kjo është reflektuar në:

- docker-compose.yml
 - auth-service/appsettings.json
-

3. Si të ekzekutoni projektin

Nga dosja kryesore e projektit:

```
cd ~/mirko-sherbimet
chmod +x scripts/run-all-services.sh
./scripts/run-all-services.sh
```

Pas fillimit, shërbimet e disponueshme janë:

- Dashboard: `http://localhost:5080`
 - Auth API: `http://localhost:5038/swagger`
 - Resource 1 health: `http://localhost:5226/health`
 - Resource 2 health: `http://localhost:5295/health`
-

4. Pika 1 - Auth Service me bcrypt dhe JWT (10 pikë)

4.1 Çfarë kontrollojnë ky dokument

Auth Service përmbush këto kërkesa:

- `/api/auth/register`
- `/api/auth/login`
- ruajtje password-i me bcrypt
- verifikim password-i me bcrypt
- gjenerim JWT me HMAC-SHA256
- skadim token-i 30 minuta

4.2 File të rëndësishme

- `auth-service/Controllers/AuthController.cs`
- `auth-service/Program.cs`
- `auth-service/Models/User.cs`
- `auth-service/Data/ApplicationDbContext.cs`
- `auth-service/appsettings.json`

4.3 Kodi kyç

Bcrypt hash

```
PasswordHash = BCrypt.Net.BCrypt.HashPassword(dto.Password)
```

Bcrypt verify

```
BCrypt.Net.BCrypt.Verify(dto.Password, user.PasswordHash)
```

Gjenerimi i JWT

```
var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jsonSettings["Key"]!));
var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

var token = new JwtSecurityToken(
    issuer: jsonSettings["Issuer"],
    audience: jsonSettings["Audience"],
    claims: claims,
    expires: DateTime.UtcNow.AddMinutes(30),
```

```
        signingCredentials: creds
    );
```

4.4 Konfigurimi i JWT

Në auth-service/appsettings.json:

```
"Jwt": {
  "Key": "THIS_IS_A_SUPER_SECRET_KEY_123456789",
  "Issuer": "auth-service",
  "Audience": "auth-service-users"
}
```

4.5 Dëshmi dhe screenshot

1. Screenshot i kodit në AuthController.cs ku shfaqet HashPassword.
2. Screenshot i kodit në AuthController.cs ku shfaqet Verify.
3. Screenshot i kodit me GenerateJwtToken dhe HmacSha256.
4. Screenshot nga <http://localhost:5080> me register/login.
5. Screenshot i token-it të kthyer nga /api/auth/login.

5. Pika 2 - Dy Resource Services që validon JWT (10 pikë)

5.1 Çfarë kontrollon ky dokument

Resource Services duhet të demonstrojnë:

- pa token -> 401
- me token valid -> 200
- me token të pavlefshëm -> 401

5.2 File të rëndësishme

- resource-service-1/Program.cs
- resource-service-2/Program.cs
- resource-service-1/appsettings.json
- resource-service-2/appsettings.json
- dashboard/Pages/ApiTest.razor

5.3 Kodi kyç

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
```

```

        ValidateAudience = true,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = builder.Configuration["Jwt:Issuer"],
        ValidAudience = builder.Configuration["Jwt:Audience"],
        IssuerSigningKey = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!))
    };
});

```

5.4 Dëshmi dhe screenshot

1. Screenshot i kodit të JWT Bearer në resource-service-1/Program.cs.
2. Screenshot i kodit të JWT Bearer në resource-service-2/Program.cs.
3. Screenshot i rezultatit 401 në /secure pa token.
4. Screenshot i rezultatit 200 në /secure me token valid.
5. Screenshot i rezultatit 401 me token të modifikuar.

6. Pika 3 - AI Module Training me Isolation Forest (10 pikë)

6.1 Çfarë kontrollon ky dokument

AI Module trajnon dhe ruan një model IsolationForest në mjedisin lokal.

6.2 File të rëndësishme

- ai-service/train.py
- ai-service/requirements.txt
- models/isolation_forest.pkl
- models/model_config.json

6.3 Kodi kyç

Në ai-service/train.py:

- Krijohen 180 mostra normale
- Krijohen 40 mostra të ndryshme normale
- Krijohen 15 mostra brute-force
- Krijohen 15 mostra impossible travel
- Trajtohet IsolationForest(contamination=0.08, random_state=42)
- Ruhet modeli me joblib.dump
- Ruhet threshold me JSON

6.4 Dëshmi dhe screenshot

1. Screenshot i kodit në `ai-service/train.py` ku ndërtohet `train_df`.
 2. Screenshot i kodit ku krijohet modeli `IsolationForest`.
 3. Screenshot i output-it të terminalit me `Model trained and saved successfully!`.
 4. Screenshot i `models/isolation_forest.pkl`.
 5. Screenshot i `models/model_config.json`.
-

7. Pika 4 - Përgjigje me shkrim (10 pikë)

7.1 File i rëndësishëm

- `docs/PERGJIGJET_SHKRIMORE.md`

7.2 Çfarë përmban

Dokumenti mbulon:

- (a) JWT vs session
- (b) bcrypt vs SHA-256
- (c) Isolation Forest
- (d) pse rate limit nuk mjafton
- (e) krahasim me Auth0 / Keycloak / Cognito

7.3 Dëshmi dhe screenshot

1. Screenshot i faqes së parë të `docs/PERGJIGJET_SHKRIMORE.md`.
 2. Screenshot i paragrafit JWT vs session.
 3. Screenshot i paragrafit bcrypt vs SHA-256.
 4. Screenshot i paragrafit Isolation Forest.
 5. Screenshot i tabelës krahasuese Auth0/Keycloak/Cognito.
-

8. Testet e plota dhe rezultatet

Për të treguar se gjithçka funksionon, përdor komandën:

```
./scripts/run-all-services.sh
```

Screenshot kryesore që duhet të bësh:

- Rezultati i plotë i komandës me 12 `passed`, 0 `failed`

Kjo është dëshmi që 40 pikët e përfshira në këtë projekt funksionojnë si duhet.

9. Lista e screenshot-ëve për prezantim

1. Momentin kur hapet `auth-service/Controllers/AuthController.cs`.
 2. Kodin `bcrypt` në `AuthController.cs`.
 3. Kodin `JWT` në `AuthController.cs`.
 4. Konfigurimin e `JWT` në `auth-service/appsettings.json`.
 5. Kodin `JWT validation` në `resource-service-1/Program.cs`.
 6. Kodin `JWT validation` në `resource-service-2/Program.cs`.
 7. `/secure` pa token me `401`.
 8. `/secure` me token valid me `200`.
 9. Outputin e `./scripts/run-all-services.sh`.
 10. Kodin `ai-service/train.py`.
 11. File `models/isolation_forest.pkl` dhe `models/model_config.json`.
 12. Dokumentin `docs/PERGJIGJET_SHKRIMORE.md`.
-

10. Plan për video-record

Renditja e demo-s:

1. Hapni VS Code dhe shfaqni strukturën e folderëve.
2. Hapni `auth-service/Controllers/AuthController.cs` dhe tregoni `bcrypt` + `JWT`.
3. Hapni `resource-service-1/Program.cs` dhe `resource-service-2/Program.cs`.
4. Hapni `ai-service/train.py`.
5. Hapni `docs/PERGJIGJET_SHKRIMORE.md`.
6. Ekzekutoni `./scripts/run-all-services.sh`.
7. Shfaqni rezultatin `12 passed, 0 failed`.
8. Shfaqni `http://localhost:5080`.
9. Shfaqni `http://localhost:5038/swagger` dhe provoni `register/login`.
10. Testoni `/secure` me dhe pa token.

Kjo rrjedhë do të jetë e qartë edhe pa koment audio, sepse çdo funksion do të shfaqet vizualisht.

11. Vërejtje për versionin aktual

Ky projekt është një version 40-pikësh. Kërkesat e plotë 100-pikëshe përfshijnë shumë funksionalitete që nuk janë implementuar në këtë repo:

- TLS me certifikatë self-signed
- Wireshark
- bllokim automatik real-time
- simulator brute-force i plotë
- impossible travel real-time me GeoIP
- dashboard real-time me SignalR

- false positive tuning

Nëse dëshironi, mund taë ndihmoj më tej për taë shtuar këto funksionalitete në një version të avancuar.

12. Përmbledhje

Ky dokument është një guide për të mbledhur dhe prezantuar:

- funksionet kryesore të Auth Service dhe JWT
- validimin e JWT te Resource Services
- trajnimin e modelit AI
- dokumentimin e përgjigjeve të kërkuara
- screenshot dhe një plan video-demo

Përdor këtë skedë për të mbledhur dëshmitë dhe për të ndërtuar raportin për prezantim.